EF Core: Deferred Execution

Concise lesson summary | Build first, execute later

1. Core Idea

Deferred Execution means that a query is built first and executed later, when its results or a scalar value are requested.

```
var query = context.Students
    .Where(s => s.Status == "Active")
    .OrderBy(s => s.StudentId);
```

At this point, the query has been composed; the student rows have not been loaded yet.

2. When Does a Query Execute?

Builds the query	Executes the query
Where(), OrderBy(), Select(), Join()	ToList(), First(), FirstOrDefault(), Single(), Count(), Any()
Usually compose the IQueryable pipeline without immediately fetching rows.	Request results or a value and trigger database execution when applicable.

Note: ToQueryString() previews SQL; it does not execute the query.

3. Avoid Calling ToList() Too Early

Wrong — filtering in memory

```
var students = context.Students.ToList();

var activeStudents = students
    .Where(s => s.Status == "Active")
    .ToList();
```

- The first ToList() loads all students into memory.
- The later Where() filters the in-memory collection, which may waste memory and database/network resources.

Correct — filtering in the database

```
var activeStudents = context.Students
    .Where(s => s.Status == "Active")
    .ToList();
```

- EF Core translates the filter into SQL WHERE.
- The database filters the rows and returns only matching students.

4. Build Queries Step by Step

Keeping an IQueryable lets you add filters and sorting before executing. This is useful for dynamic search screens.

```
var query = context.Students;

query = query.Where(s => s.Status == "Active");
query = query.OrderBy(s => s.LastName);

// Execute when results are needed
var students = query.ToList();
```

5. Connection to the Previous Lesson

- ToQueryString(): previews the SQL query shape without executing it.
- LogTo(): records EF Core commands during actual execution.

• ToList(): executes the query and materializes the returned rows.

6. Key Takeaways

```
Deferred Execution = Build first, execute later.

Where() / OrderBy()  → Compose the query
ToList()             → Execute and load rows
```

• Keep the query as IQueryable while adding database-side filters.

• Avoid early materialization if more filtering can be done by the database.

• Not every LINQ operation is deferred: operations on in-memory collections can execute immediately.